Welcome

Digital and Computer Vision

Lesson 5: Session 2

Deployment & Course Recap

Summary of the **previous** session: Final Project – Developing a Real Industry Computer Vision Application

In the previous session we have covered :

Choosing a project.
Data collection, processing, and augmentation.
Training a model using OpenCV and Deep Learning.

Summary of the current session: Deployment & Course Recap

In this session we will covered the following:

Deployment of computer vision and deep learning models
Course Recapitulation

Deploiment of computer vision and deep learning models

Deploying computer vision and deep learning models is the crucial final step that turns your project into a real-world application. It allows others to use your model via a web app, desktop interface, mobile device, or embedded system like a Raspberry Pi or Jetson Nano.

Deployment Targets

Target	Description	Tools
🖥️ **Web App**	Users access via browser	Flask, FastAPI, Streamlit, Gradio
📱 **Mobile App**	Runs on iOS/Android	TFLite, CoreML, MediaPipe
🤖 **Edge Device**	Raspberry Pi, Jetson, ESP32-CAM	TensorFlow Lite, OpenCV
☁️ **Cloud Service**	API hosted online	AWS, Google Cloud, Azure, Hugging Face Spaces
🧠 **Embedded into Software**	Integrate with custom desktop tools	PyInstaller, ElectronJS + Python backend

Model Conversion and Exporting

After training a deep learning model (typically in frameworks like TensorFlow, PyTorch, or Keras), the raw model isn't always ready for deployment.

Model conversion is the process of transforming a trained model into a format optimized for a specific platform or runtime.

Framework	Trained Model Format	Deployment Format	Conversion Tool
TensorFlow/Keras	`.h5`, `.pb`, SavedModel	`.tflite`	TFLite Converter
PyTorch	`.pt`, `.pth`	`.onnx`, `.torchscript`	TorchScript, ONNX export
ONNX	`.onnx`	ONNX Runtime, OpenVINO, TensorRT	ONNX Runtime
Scikit-learn/XGBoost	`.pkl`	Not common for mobile	joblib, pickle

Course Recapitulation - From Digital Image Processing to Computer Vision and Deep Learning

Lesson 1: Introduction to Computer Vision and Digital Image Processing.

Lesson 2: Digital Image Processing : smoothing, thresholding, edge and contours detection.

Lesson 3: Computer Vision Techniques : segmentation, feature and object detection and classification

Lesson 4: Deep Learning for Computer Vision : CNNs, TensorFlow, Face Recognition

Lesson 5: Final Project and Course Recap

Why Computer Vision

Computer Vision is a field of Artificial Intelligence (AI) that enables machines to see, interpret, and understand visual information from the world, simulating the way humans do.

It has many Industrial and scientific applications in fields like manufacturing, biomedical imaging, logistics, warehousing, and shipping industries, security, traffic monitoring, parking management, toll collection, law enforcement, customer behavior tracking, smart cities, intelligent transportation systems, Optical Character Recognition and many others.

Computer Vision Pipeline

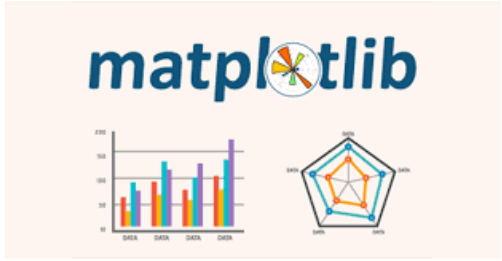
- Computer Vision relies on image processing techniques, machine learning, and deep learning to analyze and understand visual data. The basic steps include:

- 1. Image Acquisition 📷 ⟶ Capture images or video from cameras, sensors, or files.
- 2. Preprocessing 🔍 ⟶ Convert to grayscale, resize, normalize, or filter noise.
- 3. Feature Extraction 📊 ⟶ Detect edges, objects, faces, or patterns.
- 4. Object Recognition 🎯 ⟶ Use AI models to classify and recognize objects.
- 5. Decision Making 🧠 ⟶ Use AI to take actions based on visual data (e.g., detect a pedestrian and stop a car).

Principal Libraries and Tools we have used in this course

Libraries : OpenCV, NumPy, matplotlib, TensorFlow, TensorFlow Hub, face_recognition, Dlib, FaceNet

large datasets: ImageNet or COCO.



Digital Image

A digital image is essentially a two-dimensional matrix (or three-dimensional in the case of colored images), defined by the mathematical function f(x, y).

1. Pixel Values: An image is made up of pixels,

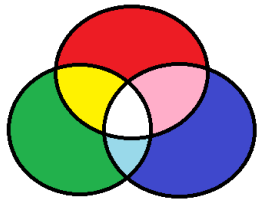
2. Digital Representation: The RGB values are usually represented as integers ranging from 0 to 255.

3. Image Matrix: The computer reads the image as a matrix of numbers, where each element in the matrix corresponds to the pixel value at that location.

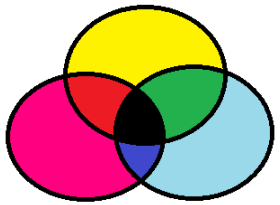
$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & f(0,2) & \dots & f(0,N-1) \\ f(1,0) & f(1,1) & f(1,2) & \dots & f(1,N-1) \\ \vdots & \vdots & \vdots & & \vdots \\ f(M-1,0) & f(M-1,1) & f(M-1,2) & \dots & f(M-1,N-1) \end{bmatrix}$$

Color Spaces

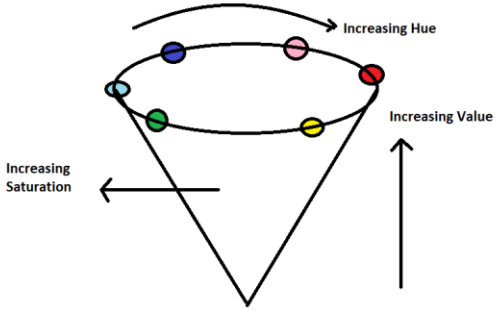
- Color spaces are a way to represent the color in the image.
- https://www.geeksforgeeks.org/color-spaces-in-opencv-python/



RGB color space



CMYK color space

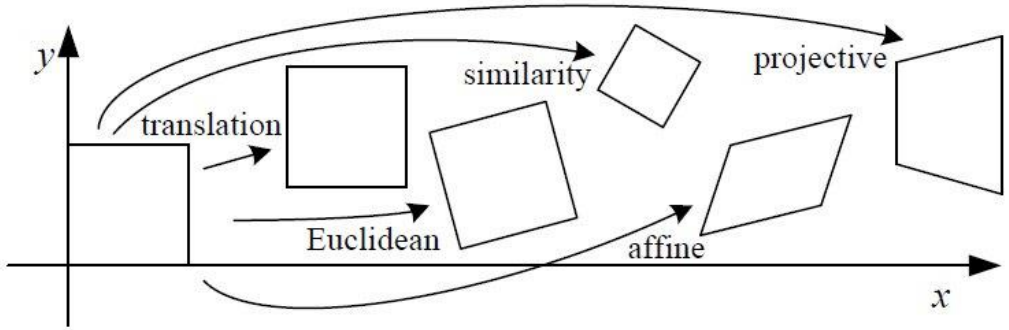


HSV color space

Digital Image Processing - Resizing images

Interpolation Method	Description	Best For
cv2.INTER_NEAREST	Nearest-neighbor without interpolation (fastest, low quality).	Simple graphics, pixel art
cv2.INTER_LINEAR	Bilinear interpolation of nearest 4 neighboring pixels (default, moderate quality).	General resizing
cv2.INTER_CUBIC	Bicubic interpolation (uses 16 pixels, higher quality but slower).	High-quality resizing
cv2.INTER_LANCZOS4	Lanczos interpolation (uses 8×8 pixel neighborhood, best quality).	High-resolution images
cv2.INTER_AREA	Area-based resampling (reduces noise, best for downscaling).	Shrinking images

Digital Image Processing - image transformations



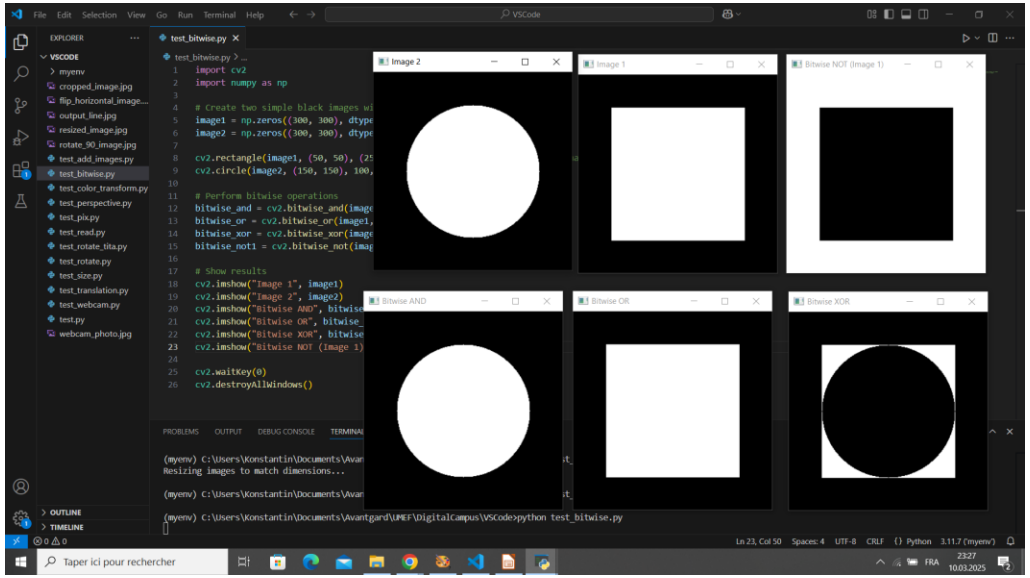
Basic set of 2D planar transformations.

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2\times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2\times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2\times 3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2\times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3\times 3}$	8	straight lines	

Szeliski, R. "Computer Vision: Algorithms and Applications." (https://szeliski.org/Book/)

Digital Image Processing – image arithmetic

Add, substract blend and bitwase operations



Digital Image Processing - Image Filtering

Images can be filtered with various filters which helps in removing noise add artistic effects or for preprocessing for CV.
The most widely used type is **convolutional filtering** where an image is modified using a filter (kernel) that is sistematically applied across the original image.

45	60	98	127	132	133	137	133
46	65	98	123	126	128	131	133
47	65	96	115	119	123	135	137
47	63	91	107	113	122	138	134
50	59	80	97	110	123	133	134
49	53	68	83	97	113	128	133
50	50	58	70	84	102	116	126
50	50	52	58	69	86	101	120

$f(x,y)$

*

0.1	0.1	0.1
0.1	0.2	0.1
0.1	0.1	0.1

$h(x,y)$

=

69	95	116	125	129	132
68	92	110	120	126	132
66	86	104	114	124	132
62	78	94	108	120	129
57	69	83	98	112	124
53	60	71	85	100	114

$g(x,y)$

Szeliski, R. "Computer Vision: Algorithms and Applications." (https://szeliski.org/Book/)

The matrix on the left is convolved with the filter in the middle to yield the matrix on the right. The light blue pixels indicate the source neighborhood for the light green destination pixel.

Digital Image Processing - Gaussian Blurring

Unlike simple averaging, Gaussian blur applies a weighted filter, giving more importance to pixels near the center. The sum of the kernel values is normalized to 1 (so brightness remains unchanged). A 3×3 Gaussian kernel might look like this:

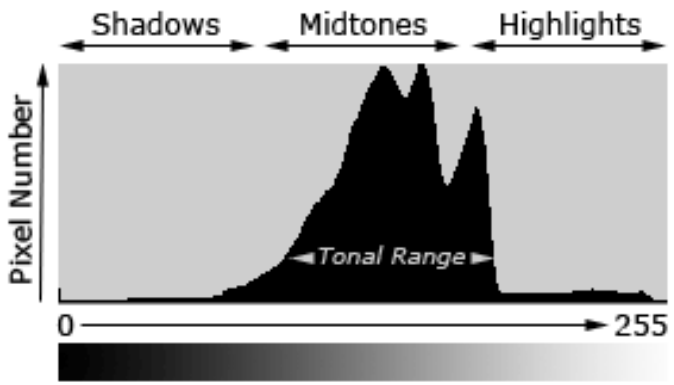
$$\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

Filter	Best for	Drawback
Averaging	General noise removal	Blurs edges
Gaussian Blur	Softening noise while keeping detail	Reduces sharpness
Median Blur	Removing salt-and-pepper noise	Slower on large images
Bilateral Filter	Noise reduction while keeping edges sharp	Computationally expensive

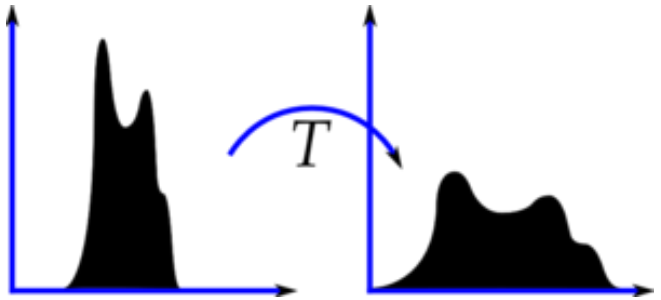
Histograms and Histogram Equalization

A histogram is a graphical representation of the distribution of pixel intensities in an image. It shows how many pixels have a specific intensity value, helping us analyze brightness, contrast, and exposure.

Understanding Image Histograms :
X-axis (Pixel Intensity) → Ranges from 0 (black) to 255 (white) (for grayscale images).
Y-axis (Frequency) → Shows the number of pixels with that intensity.



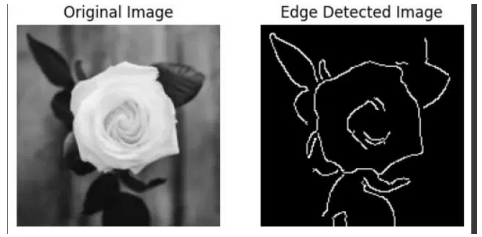
https://www.cambridgeincolour.com/tutorials/histograms1.htm



https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html

Edges

Edges in an image are boundaries where intensity or color changes significantly, usually at object borders. These edges help identify structures and shapes, ideally forming a continuous outline around objects.



https://www.geeksforgeeks.org/what-is-edge-detection-in-image-processing/

Method	Type	Noise Sensitivity	Edge Thickness	Best Use Case
Sobel	First derivative	Moderate	Thick edges	Directional edges (horizontal/vertical)
Laplacian	Second derivative	High (needs blur)	Thin edges	Fast general edge detection
Canny	Multi-step	Low (uses noise reduction)	Sharp, clean edges	Precise and accurate edge detection

Tresholding

Thresholding is a technique used in image processing to convert a grayscale image into a binary image. It is useful for object segmentation, edge detection, and shape analysis.

In thresholding, each pixel is compared to a threshold value:
If the pixel value is greater than the threshold → it is set to white (255).
If the pixel value is less than the threshold → it is set to black (0).

Thresholding Technique	Description	Best Used For
Global Thresholding	Uses a single threshold value to classify pixels as foreground or background. Example: `cv2.THRESH_BINARY`.	Simple images with uniform lighting.
Adaptive Thresholding	Computes a threshold for smaller regions of the image, adjusting to varying lighting conditions. Example: `cv2.ADAPTIVE_THRESH_MEAN_C`.	Images with non-uniform illumination.
Otsu's Thresholding	Automatically determines an optimal threshold by minimizing intra-class variance. Example: `cv2.THRESH_OTSU`.	Images with bimodal histograms (two peaks).

Contours and Contour Features

Contours are curves that join continuous points along a boundary of objects with the same color or intensity in an image. They are useful in:

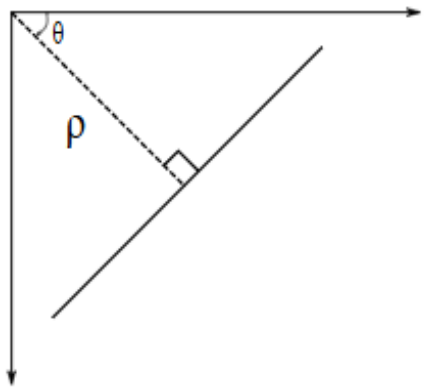
1. Object detection.
2. Shape analysis.
3. Image segmentation.
4. Measuring object properties (size, area, etc.).

1. **Contour Area** : Measures the area enclosed by the contour.
2. **Contour Perimeter** : Measures the length of the contour boundary.
3. **Contour Centroid** :Finds the center (centroid) of a contour using image moments.
4. **Bounding Box** : Finds the smallest rectangle that encloses the contour.
5. **Minimum Enclosing Circle** : Finds the smallest circle enclosing the contour.
6. **Aspect Ratio (Width-to-Height Ratio)** : Measures how elongated an object is.
7. **Convex Hull** : Finds the smallest convex shape enclosing the contour.
8. **Contour moments** : weighted averages of pixel intensities.

Hough Transform

Hough Transform is a powerful technique used to detect geometric shapes, if you can represent that shape in a mathematical form, like lines and circles, even if they are incomplete or noisy.

1. Hough Line Transform – Used to detect straight lines.



2. Hough Circle Transform – Used to detect circles.

Morphological Operations

Morphological operations like erosion and dilation are fundamental image processing techniques, especially useful in binary images (but applicable to grayscale too). They are primarily used to modify the structure or shape of objects in an image.

Erosion erodes away the boundaries of foreground object.
Dilation is just opposite of erosion. It increases the white region in the image or size of foreground object increases.

⚙️ Morphological Operations

Method	Description	Use / Task
Erosion / Dilation	Shrinks or expands object regions.	Noise removal, gap filling
Opening / Closing	Erosion followed by dilation (Opening) and vice versa (Closing).	Noise cleaning, hole filling
Morphological Gradient	Difference between dilation and erosion.	Highlights edges of objects

Computer Vision Workflow

1. Image Segmentation - separate meaningful regions from the background to focus on important parts of the image. The goal is to identify objects, boundaries, or specific areas of the real world within an image.
2. Feature Extraction and Matching - represent visual content numerically to compare or describe it. Features are the backbone of computer vision. Whether using traditional methods like SIFT, HOG, and SURF or deep learning-based CNN, features help machines "understand" images.
3. Object Classification - assign a category label to the detected object or region.
4. Image Understanding / Scene Interpretation - combine all the above to derive meaningful insights from images or video.

Example Flow:
1. Segment the road and cars from a street image.
2. Extract features from each segmented car.
3. Classify each car (e.g., sedan, SUV, truck).
4. Understand traffic patterns or detect anomalies (e.g., illegal parking, accident).

Types of algorithms for Image Segmentation

1. **Thresholding-Based Segmentation** : Uses intensity values to separate regions.
2. **Edge-Based Segmentation** : Detects object boundaries by identifying strong intensity changes.
3. **Region-Based Segmentation** : Groups pixels with similar properties (e.g., color, texture).
4. **Clustering-Based Segmentation** : Uses algorithms like K-means or Gaussian Mixture Models (GMM) to classify pixels into clusters.
5. **Deep Learning-Based Segmentation** : Uses convolutional neural networks (CNNs) for more accurate segmentation.

Algorithms of Region-Based Segmentation

🔍 Segmentation

Method	Description	Use / Task
Watershed Algorithm	Segments overlapping objects using contours as barriers.	Object separation in cluttered scenes
K-means Clustering	Partitions image into K regions based on color similarity.	Color-based segmentation
GrabCut	Interactive segmentation using graph cuts.	Foreground extraction

Types of Features in Computer Vision

1. **Low-Level Features** : Basic image properties like corners, texture, and edges.

2. **Local Features** : Local features describe distinctive regions in an image that are computed at specific keypoints detected using sophisticated algorithms.
SIFT (Scale-Invariant Feature Transform) .
SURF (Speeded-Up Robust Features).
ORB (Oriented FAST and Rotated BRIEF).
FAST (Features from Accelerated Segment Test).
BRIEF (Binary Robust Independent Elementary Features).

https://docs.opencv.org/4.x/df/d54/tutorial_py_features_meaning.html

3. **Global Features** : These features describe the entire image rather than specific points.
Histogram-based features – Used for color-based image recognition.
Shape descriptors – Used for object recognition based on shape.
HOG (Histogram of Oriented Gradients) – Used for detecting objects like pedestrians.

4. **Deep Learning-Based Features** : Deep learning extracts high-level features automatically using convolutional neural networks (CNNs).

Features workflow - Detection, Description and Matching

Features detection, description and matching are fundamental steps in computer vision tasks.

1. Features detection is the process of identifying features in an image.

2. Once identified we have to assigning a numerical descriptor (a feature vector) to each detected feature. This descriptor encodes important properties so that it can be matched across different images.

3. Finaly, we can compare feature descriptors between two images using feature matching algorithms.

Feature algorithms

🔑 Feature Detection & Matching

Method	Description	Use / Task
Harris Corner	Detects corners in an image.	Motion tracking, image registration
ORB, SIFT, SURF	Detects and matches features between images.	Object tracking, stitching, recognition

Object Classification

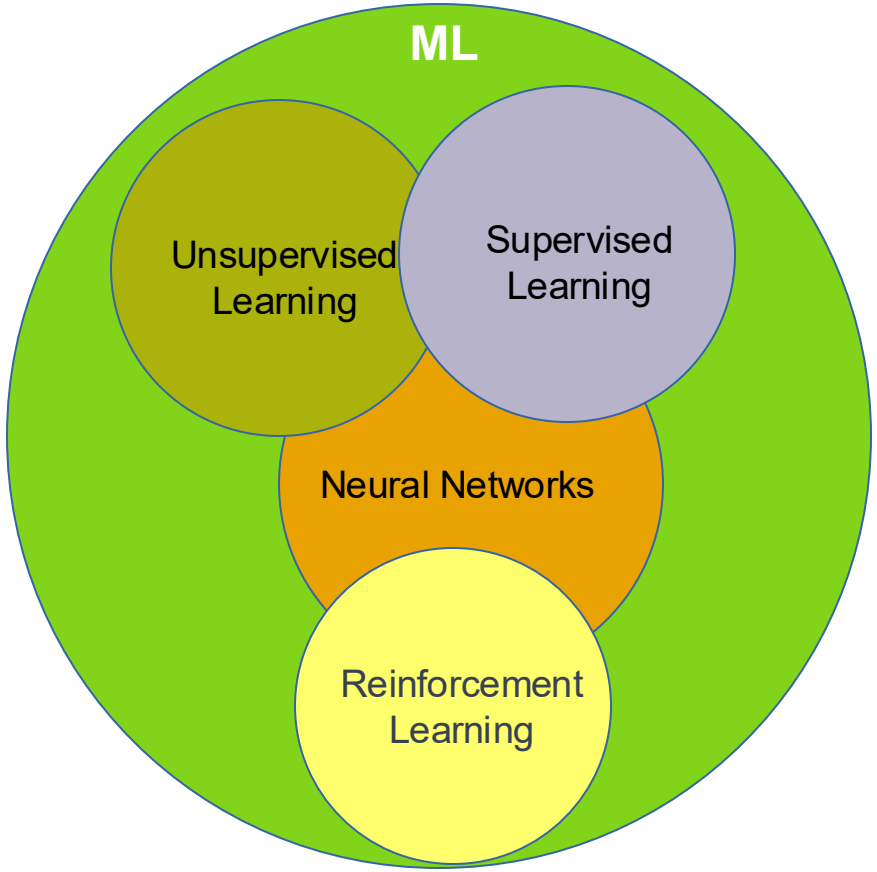
Classical ML: Haar Cascades, HOG + SVM, KNN

Deep Learning:

CNNs (TensorFlow) for image classification.

Fine-tuning on custom data for specific tasks.

Machine Learning



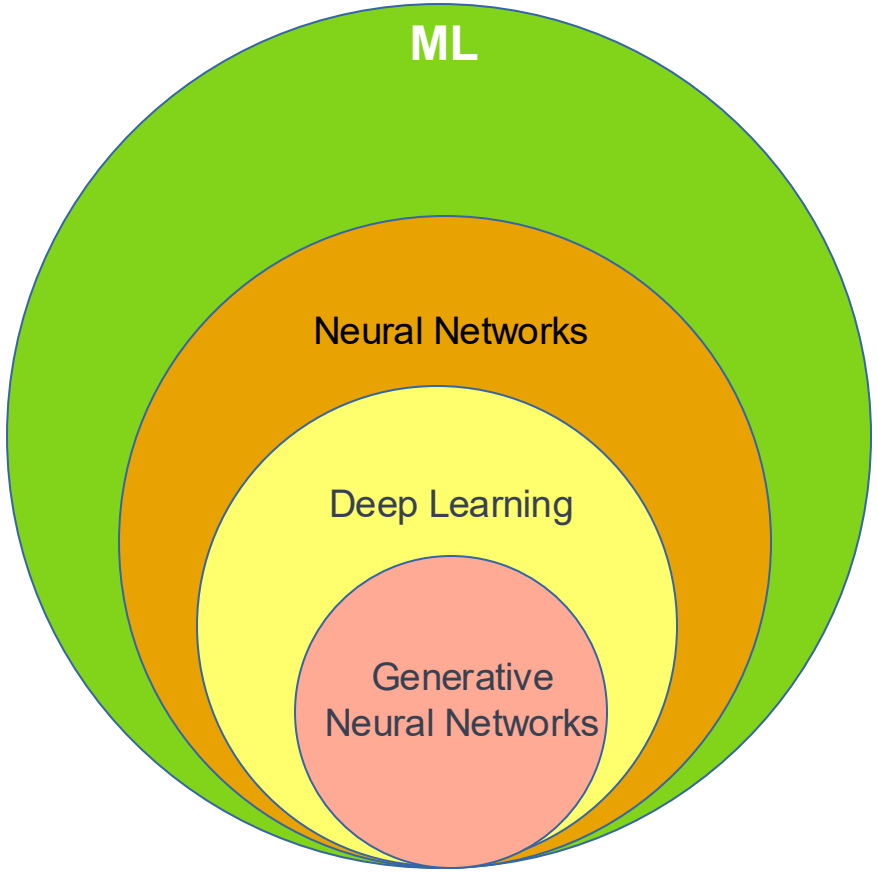
Unsupervised Learning: Learning from unlabeled data, finding patterns and relationships within the data.

Supervised Learning: Learning from labeled data, where the algorithm is trained on input-output pairs.

Reinforcement Learning: Learning through trial and error, where the algorithm receives feedback in the form of rewards or penalties.

Neural Networks: Systems inspired by the structure and function of the human brain, composed of interconnected nodes (neurons) organized in layers.

Neural Networks



Deep Learning: involves the development and application of artificial neural networks with multiple layers (deep neural networks) to analyze and extract patterns from large sets of data (Big Data).

Generative Neural Networks: focus on generating new data instances that resemble a given training dataset.

Deep Learning

Deep Learning is a subfield of machine learning that focuses on algorithms inspired by the structure and function of the human brain, known as artificial neural networks. It involves training models with many layers (hence "deep") to automatically learn complex patterns and representations from large amounts of data.

Key Characteristics:
Neural Networks: Deep learning models are built using layers of neurons (nodes) that mimic how the brain processes information.

Multiple Layers: These networks have multiple layers between input and output (e.g., Convolutional Layers, Recurrent Layers, Fully Connected Layers).

Feature Learning: Automatically learns features from raw data (e.g., images, text, audio) — no need for manual feature engineering.

High Performance: Particularly powerful for tasks involving vision, speech, natural language, and autonomous systems.

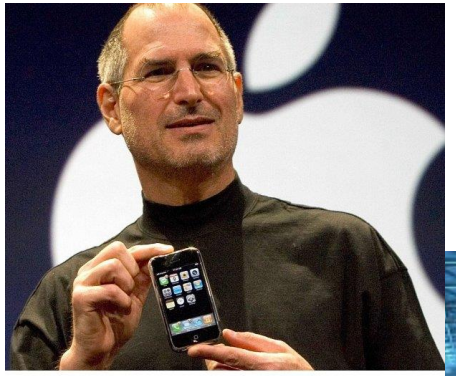
Fundamental Difference between DNN Old Neural Networks

Neurons are structured in multiple layers with specific functions similar to the human brain!

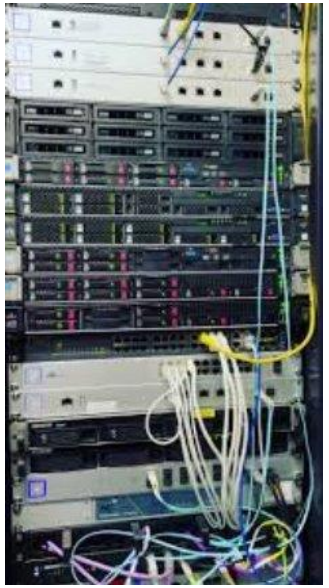
Eureka! K. Startchev and NightCafe

Deep Learning

We have described a neural network capable of using a large number of neurons, but we also need a large number of data and enormous computing power to train this network.



Netscape (1994)
Smatphone (2007)
Storage space 2003
Servers (Amazon) (2006)



graphic processor optimised for deep learning

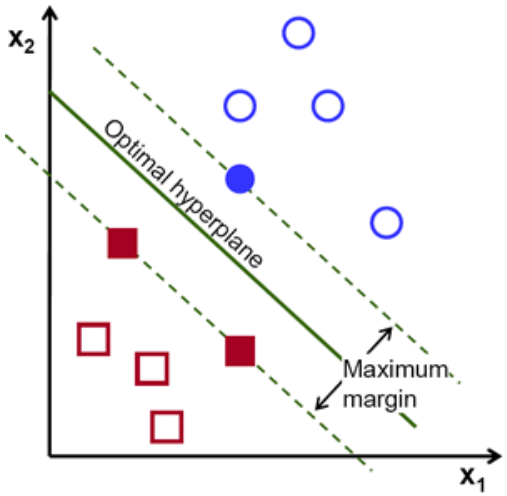
processor of 320000 neurons for computer vision application

Model

Definition:
A model is a function (or set of functions) that maps input data to output predictions using parameters learned from training data.

Think of it like this:

"Model" = Mathematical Machine
Input: Features (e.g., pixel values, numbers, text)
Process: Applies weights, biases, and activations (these are parameters)
Output: A prediction (e.g., digit label, cat vs. dog, sentiment, etc.)



DNN – Basic Workflow

1. **Define the model**: defining the entire architecture of the model

2. **Compile the model**: tells the model how to learn (how to adjust its internal weights during training and how to evaluate its performance)

3. **Train the model**: optimise parameters.

4. **Predict**.

In this session we have covered:

Deployment of computer vision and deep learning models
Course Recapitulation

THANKS !!!